



Advanced OpenMP

Newer Features



Overview

- User defined reductions
- Reduction extensions
- Scan operations
- Construct cancellation
- SIMD directives
- Thread affinity
- Masked construct
- Linear clause
- Loop transformations
- Synchronisation hints
- Metadirectives

Not an exhaustive list, nor an in-depth coverage of these topics – the OpenMP spec is now enormous!

Not all compilers may support all these features.

Specification is now very hard to read – often more helpful to consult the examples document at openmp.org

User defined reductions

- Built-in reductions are limited to basic data types (or arrays of these) and a restricted list of operators
 - does now include **min** and **max** in C/C++!
- Use **declare reduction** directive to define new reduction operators
- New operators can then be used in reduction clause.

Version 4.0

```
#pragma omp declare reduction (reduction-identifier :  
typename-list : combiner) [identity(identity-expr)]
```

- **reduction-identifier** gives a name to the operator
 - Can be overloaded for different types
 - Can be redefined in inner scopes
- **typename-list** is a list of types to which it applies
- **combiner** expression specifies how to combine values
- **identity** can specify the identity value of the operator
 - Can be an expression or a brace initializer

Example: merging vectors

```
#pragma omp declare reduction (merge : std::vector<int>
: omp_out.insert(omp_out.end(), omp_in.begin(), omp_in.end()))
```

- Private copies created for a reduction are initialized to the identity that was specified for the operator and type
 - Default identity defined if identity clause not present
- Compiler uses combiner to combine private copies
- **omp_out** refers to private copy that holds combined values
- **omp_in** refers to the other private copy
- Can now use **merge** as a reduction operator.

Reduction extensions

- Can force a reduction to happen at a specified point inside a parallel region with the **scope** construct:

```
#pragma omp parallel shared(a) private(b)
{
    #pragma omp single
    {
        a = 0.0;
    }
    #pragma omp scope reduction(+:a)
    {
        a += foo();
    } // reduction happens here
    b = 2 * a;
}
```

Version 5.1

- Also possible to do reductions on private variables inside a parallel region.
 - each private copy is combined with all the updates to the reduction variable

```
#pragma omp parallel private(a,b)
{
    a = 0.0;
    #pragma omp for reduction(+:a)
    for (i=0; i<N; i++)
        a += foo();
} // all private copies get updated
b = 2 * a;
}
```

Version 6.0

Scan operations

- As well as reductions, OpenMP now has support for prefix scan
- For example, as well as computing the total sum with an addition reduction, we can also compute all the partial sums across all the iterations
 - can be inclusive (i.e. k^{th} value includes k^{th} iteration) or exclusive (only values up to iteration $k-1$)

```
!$omp parallel do reduction(inscan,+:x)
do k = 1, n
    x = x + a(k)
!$omp scan inclusive(x)
    b(k) = x
end do
```

Version 5.0

Construct cancellation

- Clean way to signal early termination of an OpenMP construct.
 - one thread signals
 - other threads jump to the end of the construct

```
!$omp cancel construct [if (expr)]  
#pragma omp cancel construct [if (expr)]
```

where *construct* is **parallel**, **sections**, **do/for** or **taskgroup** cancels the construct

Version 4.0

```
!$omp cancellation point construct  
#pragma omp cancellation point construct
```

checks for cancellation (also happens implicitly at cancel directive, barriers etc.)

Example

```
!$omp parallel do private(eureka)
do i=1,n
    eureka = testing(i,...)
!$omp cancel parallel if(eureka)
end do
```

- First thread for which **eureka** is true will cancel the parallel region and exit.
- Other threads exit next time they hit the **cancel** directive

Portable SIMD directives

- Many compilers support SIMD directives to aid vectorisation of loops.
 - compiler can struggle to generate SIMD code without these
- OpenMP 4.0 provides a standardised set
- Use **simd** directive to indicate a loop should be SIMDized

Version 4.0

```
#pragma omp simd [clauses]
```

```
!$omp simd [clauses]
```

- Executes iterations of following loop in SIMD chunks
- Loop is not divided across threads
- SIMD chunk is set of iterations executed concurrently by SIMD lanes

- Clauses control data environment, how loop is partitioned
- **safelen(length)** limits the number of iterations in a SIMD chunk.
- **linear** lists variables with a linear relationship to the iteration space
- **aligned** specifies byte alignments of a list of variables
- **private**, **lastprivate**, **reduction** and **collapse** have usual meanings.
- Also **declare simd** directive to generate SIMDised versions of functions.
- Can be combined with loop constructs (parallelise and SIMDise)

Affinity environment

- Since many systems are now NUMA and SMT, placement of threads on the hardware can have a big effect on performance.
- Increased choices for `OMP_PROC_BIND`
- Can still specify `true` or `false`
- Can now provide a list (possible item values: `master`, `close` or `spread`) to specify how to bind parallel regions at different nesting levels.
- Added `OMP_PLACES` environment variable
- Can specify abstract names including threads, cores and sockets
- Can specify an explicit ordered list of places
- Place numbering is implementation defined
- Good for nested parallel regions
- For MPI + OpenMP needs a mixture of this and batch scheduler to get binding correct

Version 4.0

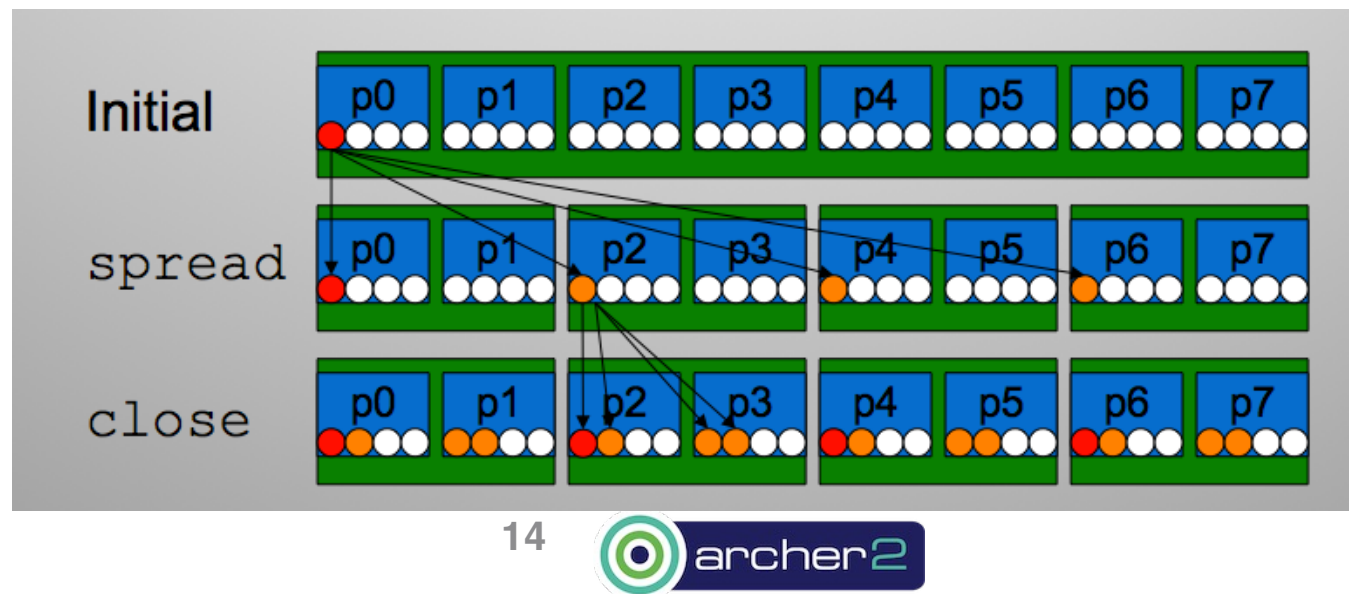
Example

- Two levels of nested parallel regions
- Hardware with 8 cores and 4 hardware threads per core

```
export OMP_NUM_THREADS=4,4
```

```
export OMP_PLACES=threads
```

```
export OMP_PROC_BIND="spread,close"
```



Masked construct

- The **master** construct has been deprecated
- It will still work, but has been renamed **masked**
- Terminology for thread 0 has changed too: *master thread* -> *primary thread*
- Now possible to select threads other than thread 0 to execute the block, with the **filter** clause.
- If the argument to the **filter** clause matches the number of the encountering thread, then that thread will execute the block
 - if more than one thread matches, they will all execute the block

Version 5.1

Masked example

- Execute block of code only on even numbered threads

```
#pragma omp parallel
{
  int includeme;
  if (omp_get_thread_num()%2 == 0) {
    includeme = omp_get_thread_num();
  } else {
    includeme = -1;
  }
  #pragma omp masked filter(includeme)
  {
    // block of code
  }
}
```


Linear clause

- An *induction* variable is a variable in a loop which is incremented by a constant every iteration
- Can prevent straightforward parallelisation of loops
 - refactoring usually trivial, but can be unwanted/tedious
- Can now be handled with **linear** clause on **do/for** construct

```
j=0;  
#pragma omp parallel for linear(j:1)  
for(i=0; i<N; i+=2){  
    b[j] = a[i] * 2.0f;  
    j++;  
}
```

Version 4.5

Loop transformations

- Many compilers support programmer control of loop transformations (e.g. unrolling, tiling, interchanging, reversing) via compiler-specific directives
- OpenMP now supports a portable mechanism for these transformations (and combinations of them).

Version 5.1

```
#pragma omp unroll full
for(i=0; i<4; i++){
    b[i] = 2.0 * a[i];
}
```

Fully unrolls a sequential loop

```
!$omp parallel do
!$omp tile sizes(5,16)
    do i = 1, 100
        do j = 1, 128
            A(j,i) = j*1000 + i
        end do
    end do
```

Tiles loop nest into 5 x 16 blocks and distributes blocks across threads

Synchronisation hints

- Can give hints about the expected use of synchronisation constructs (critical, atomics, locks) to assist optimisation of these.
- *contended* if we expect threads to wait for each other frequently, or *uncontended* if not.
- *speculative* if using transactional memory might be beneficial, or *nonspeculative* if not.
- **critical** and **atomic** constructs take a **hint** clause, e.g.

```
#pragma omp atomic hint(omp_sync_hint_uncontended)
    a[i]++;
```

- Can initialise locks with **omp_init_lock_with_hint()** instead of **omp_init_lock()**

Version 4.0/4.5

Monotonic/nonmonotonic schedules

- Older versions of OpenMP mandated that loop chunks are assigned to threads in their logical execution order
 - e.g. for a dynamic schedule, the k^{th} chunk must be assigned to a thread before the $k+1^{\text{th}}$ chunk.
- This restriction has been relaxed: chunks can be assigned in any order
 - can improve data affinity, and reduce synchronisation overheads
- Can enable/disable monotonicity if you want to, e.g.

```
!$omp parallel do schedule(monotonic:dynamic)
```

```
!$omp parallel do schedule(nonmonotonic:dynamic)
```

- Default is monotonic for **static** schedule kind, nonmonotonic otherwise.

Version 5.0

Metadirectives

- Syntax that supports runtime choice of OpenMP directives without having to explicitly write conditional code.
- Can do tests based on program variables, or on the type of offloading device, or on the ISA of the CPU.
- Can also create code variants and select which is used.

```
#pragma omp metadirective \  
when(user={condition(unbal)}): \  
for schedule(guided)) \  
otherwise(for schedule(static))  
for (i=0; i<N; i++){  
    ...  
}
```

```
!$omp metadirective &  
!$omp when(user={condition(unbal)}):&  
!$omp do schedule(guided)) &  
!$omp otherwise(do schedule(static))  
do i=1,N  
    ...  
end do
```

Version 5.0

Reusing this material



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

This means you are free to copy and redistribute the material and adapt and build on the material under the following terms: You must give appropriate credit, provide a link to the license and indicate if changes were made. If you adapt or build on the material you must distribute your work under the same license as the original.

Note that this presentation contains images owned by others. Please seek their permission before reusing these images.

